

Roteiro de Laboratório: Praticando Governança de TI (COBIT 2019) com Python

Curso: Sistemas de Informação

Disciplina: Gestão de Sistemas de Informação

Objetivos:

Diferenciar Governança de Gestão na prática, simulando a tomada de decisão do Conselho (EDM) versus a execução da Diretoria (APO/BAI/DSS/MEA).

Implementar o Princípio 1 do COBIT (Prover Valor) através de um algoritmo que calcula o balanço entre Benefícios, Custos e Riscos.

Simular Fatores de Desenho (Customização) ajustando o peso dos domínios com base no perfil de risco da organização.

1. Cenário Prático: Auditoria e Alinhamento Estratégico com COBIT 2019

Contexto: Você foi contratado como Consultor(a) de Governança e Sistemas de Informação para a *TechGov Solutions*. A diretoria executiva da empresa está preocupada porque a TI é vista como uma "caixa preta" pelos executivos tradicionais, tornando difícil medir o desempenho real e alinhar os investimentos tecnológicos às metas estratégicas do negócio.

Sua Missão: Desenvolver um protótipo de sistema em Python que funcione como um **Motor de Decisão e Auditoria**. O script deverá processar dados brutos de indicadores operacionais e estratégicos da empresa, categorizá-los automaticamente dentro dos domínios do framework COBIT 2019, e gerar um relatório automatizado consolidado para o Conselho de Administração (Board de Diretores).

2. Os Dados de Entrada

Abaixo está o conjunto de dados brutos que o seu script deve carregar e processar. Ele representa o estado atual de 6 indicadores críticos coletados pela auditoria interna.

Dica de Implementação: Você pode estruturar esses dados diretamente como um dicionário de dicionários no seu código ou ler a partir de um arquivo externo.

```
dados_auditoria = {  
    "IND_01": {  
        "nome": "Nível de Transparencia para Stakeholders externos",  
        "valor_atual": 4.2,  
        "meta": 4.5  
    },  
    "IND_02": {  
        "nome": "Alinhamento do Orcamento de TI com a Estratégia",
```

```

    "valor_atual": 4.5,
    "meta": 4.0
  },
  "IND_03": {
    "nome": "Taxa de Sucesso no Gerenciamento de Projetos e Mudancas",
    "valor_atual": 3.8,
    "meta": 4.0
  },
  "IND_04": {
    "nome": "Tempo de Resposta do Service Desk em Operacoes Diarias",
    "valor_atual": 2.5,
    "meta": 4.0
  },
  "IND_05": {
    "nome": "Conformidade com Leis de Privacidade e Auditoria Interna",
    "valor_atual": 2.0,
    "meta": 4.5
  },
  "IND_06": {
    "nome": "Retorno de Investimento (ROI) em Iniciativas de Inovacao",
    "valor_atual": 4.8,
    "meta": 3.5
  }
}

```

3. Regras de Negócio e Requisitos do Sistema

O seu script deverá processar a estrutura acima seguindo três pilares fundamentais do COBIT 2019 presentes em nosso material de aula:

3.1. Separação entre Governança e Gestão

O algoritmo de classificação deve mapear automaticamente cada indicador para um dos 5 Domínios do COBIT 2019 com base em palavras-chave encontradas no nome do indicador:

- EDM (Avaliar, Direcionar e Monitorar): Indicadores ligados ao Conselho, governança de alto nível e partes interessadas.

- APO (Alinhar, Planejar e Organizar): Indicadores de estratégia, orçamento, inovação e recursos humanos.
- BAI (Construir, Adquirir e Implementar): Indicadores de projetos, requisitos e gestão de mudanças.
- DSS (Entregar, Servir e Suportar): Indicadores de operações diárias, service desk e segurança da informação.
- MEA (Monitorar, Avaliar e Analisar): Indicadores de conformidade legal, auditorias e desempenho regulatório.

3.2. Análise de Atingimento de Metas (Gap Analysis)

O sistema deve calcular a diferença entre o valor_atual e a meta de cada indicador.

- Se o valor_atual for maior ou igual à meta, o indicador está "Em Conformidade".
- Se for menor, o indicador está "Abaixo da Meta (Crítico)".

3.3. Fatores de Desenho (Customização do Peso)

A governança eficaz deve ser adaptada ao perfil da empresa (Princípio 5). Seu script deve permitir selecionar o Foco Estratégico da organização antes de gerar o resultado final, aplicando pesos diferentes para calcular a Nota Global de Maturidade de TI (de 0 a 5):

- Foco em Inovação: Dá mais relevância e peso para as notas dos domínios APO e BAI.
- Foco em Segurança e Conformidade: Dá mais relevância e peso para os domínios DSS e MEA.

4. Resultado Esperado (Output no Terminal)

O script deve imprimir na tela um relatório limpo e executivo dividido em duas seções claras:

1. Visão do Board (Governança - EDM): Situação dos objetivos estratégicos da alta administração e a nota consolidada de maturidade da empresa ajustada pelo Fator de Desenho escolhido.
2. Visão Operacional (Gestão - APO/BAI/DSS/MEA): Uma lista alertando os gerentes de TI sobre quais processos diários e operacionais falharam em atingir a meta e precisam de planos de ação imediatos.

Atividade 1: Classificação Automatizada de Domínios COBIT 2019

Como vimos em aula, o COBIT separa claramente as responsabilidades de Governança e Gestão. Para automatizar a nossa auditoria, sua primeira tarefa é criar uma função que analise o nome de qualquer indicador e determine a qual dos 5 domínios do framework ele pertence.

Sua Tarefa: Complete a função `classificar_dominio_cobit` abaixo. Ela deve usar estruturas condicionais (`if/elif/else`) para varrer o nome do indicador em busca de palavras-chave estratégicas e retornar a sigla do domínio correspondente (EDM, APO, BAI, DSS ou MEA).

```

def classificar_dominio_cobit(indicador_nome):
    """
    Mapeia strings de indicadores para os 5 domínios do COBIT 2019.
    """

    indicador = indicador_nome.lower()

    # 1. Domínio de Governança (EDM)
    if "board" in indicador or "transparencia" in indicador or "meta_conselho" in indicador:
        return "EDM"

    # 2. Domínios de Gestão (APO, BAI, DSS, MEA)
    elif "estrategia" in indicador or "orcamento" in indicador or "arquitetura" in indicador:
        return "APO"

    elif "projeto" in indicador or "requisito" in indicador or "mudanca" in indicador:
        return "BAI"

    elif "operacao" in indicador or "suporte" in indicador or "seguranca" in indicador:
        return "DSS"

    elif "auditoria" in indicador or "conformidade" in indicador or "leis" in indicador:
        return "MEA"

    else:
        return "Nao Identificado"

# --- Área de Teste Local ---
# Use os prints abaixo para validar se sua lógica de captura está correta:
print(classificar_dominio_cobit("Seguranca de Operacoes Diarias")) # Esperado: DSS
print(classificar_dominio_cobit("Nivel de Transparencia do Board")) # Esperado: EDM

```

Atividade 2: Filtro de Viabilidade e Geração de Valor (Princípio 1)

O Princípio 1 do COBIT estabelece que qualquer sistema de governança deve focar em Prover Valor. Na prática, isso exige um equilíbrio constante entre a entrega de benefícios, a otimização dos custos e a mitigação dos riscos associados.

Sua Tarefa: Implemente uma função que simule a aprovação automática de novos investimentos de TI. O algoritmo deve receber notas de 0 a 10 para os três pilares e aplicar os pesos recomendados

para calcular um score final. Se o projeto pontuar abaixo de 5.0, ele deve ser barrado por desalinhamento estratégico.

```
def avaliar_geracao_de_valor(beneficios, custos, riscos):  
    """  
    Calcula o score de valor baseado no Princípio 1 do COBIT.  
    Pesos sugeridos: Benefícios (50%), Custos (20%), Riscos (30%).  
    """  
  
    # Cálculo do score considerando os pesos regulamentados  
    score_valor = (beneficios * 0.5) - (custos * 0.2) - (riscos * 0.3)  
  
    if score_valor >= 5.0:  
        status = "Aprovado pelo Conselho (Gera Valor Alinhado)"  
    else:  
        status = "Rejeitado: Alto risco/custo ou baixo retorno estratégico"  
  
    return score_valor, status  
  
# --- Área de Teste Local ---  
  
# Simulação de um projeto de migração de infraestrutura para Nuvem:  
score, msg = avaliar_geracao_de_valor(beneficios=9.0, custos=4.0, riscos=3.0)  
print(f"Score do Projeto: {score:.2f} | Decisão do Board: {msg}")
```

Atividade 3: Customização por Fatores de Desenho (*Design Factors*)

O Princípio 5 do COBIT dita que o sistema de governança deve ser Adaptado às Necessidades da empresa. Organizações não são iguais: uma *Fintech* exige foco massivo em segurança e conformidade, enquanto uma *Startup* de jogos foca prioritariamente em inovação rápida. Os *Design Factors* alteram o peso que cada domínio tem no cálculo da maturidade corporativa.

Sua Tarefa: Analise a função de customização abaixo. Você deve compreender como o dicionário de pesos muda dinamicamente com base no parâmetro `fator_foco`. Teste a função utilizando os dados consolidados da última auditoria.

```
def calcular_maturidade_customizada(pontuacoes_dominios, fator_foco):  
    """  
    Ajusta o peso dos domínios com base no Fator de Desenho escolhido.  
    pontuacoes_dominios: dicionário contendo as médias obtidas em cada domínio (0 a 5).  
    """
```

```
fator_foco: string ('seguranca', 'inovacao' ou 'padrao').
"""

if fator_foco == 'seguranca':
    # Dá maior importância a DSS (Operação/Segurança) e MEA (Conformidade/Auditoria)
    pesos = {"EDM": 0.20, "APO": 0.10, "BAI": 0.10, "DSS": 0.30, "MEA": 0.30}
elif fator_foco == 'inovacao':
    # Dá maior importância a APO (Estratégia) e BAI (Construção/Projetos)
    pesos = {"EDM": 0.15, "APO": 0.30, "BAI": 0.35, "DSS": 0.10, "MEA": 0.10}
else:
    # Distribuição linear e homogênea (Cenário Padrão)
    pesos = {"EDM": 0.20, "APO": 0.20, "BAI": 0.20, "DSS": 0.20, "MEA": 0.20}

# Multiplica a nota de cada domínio pelo seu respectivo peso de foco
nota_final = sum(pontuacoes_dominios[dom] * pesos[dom] for dom in pesos)
return nota_final

# --- Área de Teste Local ---

# Médias brutas simuladas da situação atual da TI (Notas de 0 a 5)
auditoria_atual = {"EDM": 4.0, "APO": 4.5, "BAI": 4.2, "DSS": 2.5, "MEA": 2.0}

print(f"Maturidade se o foco for INOVAÇÃO: {calcular_maturidade_customizada(auditoria_atual,
'inovacao'):.2f}")

print(f"Maturidade se o foco for SEGURANÇA: {calcular_maturidade_customizada(auditoria_atual, 'seguranca'):.2f}")
```

Desafio de Integração Final (main.py)

Para consolidar o laboratório, você deve criar o script principal do sistema. Una as funções desenvolvidas nos exercícios anteriores para processar a estrutura de dados `dados_auditoria` fornecida no cenário inicial do laboratório.

Seu programa final deverá:

1. **Mapear e Agrupar:** Iterar pelo dicionário de dados brutos, usar a função do *Exercício 1* para descobrir o domínio de cada indicador e calcular a nota média real obtida em cada um dos 5 domínios.

2. **Executar a Análise de Gaps:** Identificar e listar individualmente quais indicadores operacionais (Gestão) ficaram com o `valor_atual` abaixo da `meta`.
3. **Aplicar a Visão de Negócio:** Solicitar que o usuário escolha o foco estratégico da empresa através do terminal (`inovacao` ou `seguranca`) e gerar o Índice de Maturidade Geral utilizando o motor do *Exercício 3*.
4. **Imprimir o Relatório Executivo:** Renderizar os resultados organizados na tela, dividindo explicitamente o que é relatório de Governança (Board/EDM) e o que é relatório de Gestão (Gerências/Operações).